

P0947R1

Another take on Modules

Published Proposal, 6 March 2018

This version:

<http://wg21.link/p0947r1>

Author:

[Richard Smith](#) (Google)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper discusses a set of extensions to C++ to support build scalability and library encapsulation.

Table of Contents

1	Revision history
2	Overview
2.1	Background and motivation
2.1.1	Why not the Modules TS?
2.2	Design goals
2.3	Design principles
3	Basics
3.1	Modules
3.1.1	Module ownership
3.2	Translation unit structure
3.2.1	The <i>module-declaration</i>
3.2.2	<i>import-declarations</i>
3.3	Module partitions
4	Imports
4.1	Import semantics
4.2	Transitive visibility of imports
4.2.1	Private imports
4.2.2	Exported imports
4.2.3	Public imports
4.3	Preprocessor impact

5	Exports
5.1	Name export
5.2	Macro export
6	Legacy headers
6.1	Legacy header units
6.2	<code>#include</code> translation
6.3	Compilation model
7	Low-level design details
7.1	Templates and two-phase name lookup
8	Acknowledgements
	Index
	Terms defined by this specification
	References
	Informative References

§ 1. Revision history

Changes since R0:

- `export import` no longer re-exports macros, and `#export` is now orthogonal functionality

§ 2. Overview

§ 2.1. Background and motivation

C++ is used for many of the world's largest software systems. But as systems scale upwards, a number of problems emerge that C++ does not provide adequate tools to tackle. We particularly note these problems:

- Distinct components cannot be adequately isolated from one another; implementation details leak across component boundaries, and users are tempted to break encapsulation by redeclaring entities owned by other components.
- Code does not have hermetic semantics: the meaning of an interface can depend on declarations and macros exposed by the user of that interface prior to its inclusion.
- Build performance grows near-quadratically, because in practice source files transitively depend upon and `#include` an approximately constant fraction of all components' interfaces.

The commonality between these problems is that the C++ model for providing interfaces is by redeclaring those interfaces in every source file, typically by textual inclusion.

These problems have been considered by several groups and individuals before; notably, the C++ Modules Technical Specification ([\[N4720\]](#)) provides one possible direction to resolve these issues. The design in this paper is based on a different prioritization of goals than those of the Modules TS, resulting in different choices being made in various key areas, but we follow the Modules TS in all cases where our priorities do not necessitate a different decision. We are grateful to all who have contributed to the Modules TS for providing a framework of terminology and ideas for us to draw from within this proposal.

§ 2.1.1. Why not the Modules TS?

A number of prior papers have raised design concerns with the Modules TS. We believe these concerns largely stem from the design of the Modules TS being based on a different prioritization of goals than those of the parties raising the concerns. However, we believe that it is possible to address nearly all of the raised concerns without compromising the fundamental goals of the modules effort, and this proposal seeks to do so. Specifically, we would draw the reader's attention to:

- [\[P0273R0\]](#) (by the author of this paper and others) raises a collection of concerns; we address all concerns therein other than "Module names as strings", to which the committee has expressed objections.
- [\[P0678R0\]](#) describes Bloomberg business requirements for modules. We believe this proposal addresses the described requirements, which are very similar to our own, as we provide an additive, interoperable approach to modules that provides an incremental path to adoption.
- [\[P0713R0\]](#), [\[P0774R0\]](#) raise the concern that a human reader should be able to identify whether a source file is a module unit by inspecting its initial contents, although they describe different solutions to the problem. This proposal addresses that concern: the *module-declaration* is always at the start of the translation unit if present.
- [\[P0775R0\]](#) describes the problem of permitting the interface of a module to be split across multiple source files. This proposal introduces module partitions to solve that problem and others.
- [\[P0795R0\]](#) describes compatibility problems caused by the introduction of the `module` keyword. Further research has indicated that the identifiers `module` and `import` are both in widespread usage in multiple codebases, and in some cases are part of immutable APIs. This proposal avoids breaking such code by treating these identifiers as context-sensitive keywords. This is made possible by requiring a specific ordering of the *module-declaration* and *import-declarations* within a translation unit.
- [\[P0841R0\]](#) describes a set of concrete problems that would prevent the Modules TS from being used as-is by Apple. We believe this proposal resolves all the problems described in that paper; the modules proposal we describe here provides as a subset a system very similar to the Clang header modules system currently in use by Apple.
- [\[N4697\]](#) contains the accumulated ballot comments on the Modules TS, reflecting many of the design concerns raised above, which we believe are well addressed by this proposal.

§ 2.2. Design goals

In order to support scalable software development, this paper aims to do the following:

- Provide a mechanism to allow the interface of a translation unit to be controlled and exposed to other translation units.
- Ensure that natural implementation strategies for importation of such an interface have a compile-time cost that is linear in the size of the interface that is used (rather than linear in the size of the transitive interface that is exposed, as is the case in current C++).
- Ensure interfaces are hermetic: the exported interface should not depend in any way on earlier-declared names and macros.
- Ensure components are encapsulated: prevent code outside a component from "reaching into" that component and accessing its internals.
- Allow components to control the interface they expose to other components, and in particular provide clear boundaries around which parts of the implementation are exposed to client code and which are not.
- Support incremental adoption of the features within this proposal, where it is expected that in some cases it will take decades for all clients of a component to transition to the new features, and the software must remain maintainable in the interim.
- Support continued use of legacy code that cannot ever reasonably transition to these features, without resorting to the myriad problems of textual inclusion.

- Arrange the language features to facilitate basic understanding of C++ code by tools that are not compilers.
- Do not change the fundamentals of C++, except as necessary to support the above goals.

§ 2.3. Design principles

The extensions described in this paper aim to conform to these design principles:

- *Be consistent and general.* New language features should integrate into the existing language, and should consistently support all facets of the existing language equally. This proposal directly supports exporting and importing all constructs that participate in C++ interfaces, including macros.
- *Be orthogonal.* Distinct concepts and functionality should not be unnecessarily coupled, and programmers should be given the flexibility to choose when to use features together. This proposal does not couple module boundaries to translation unit boundaries, instead permitting modules to span as many few or many translation units as is suitable for the project. Likewise, whether a translation unit provides part of the interface of a module is not coupled to whether a translation unit is importable, permitting importation of implementation details within a module.
- *Old code and new code are both critical.* The success of C++ is in part due to transparently supporting a huge subset of existing C code, and allowing existing code to immediately take advantage of new features. To be adopted, any language extension that aims to fundamentally change how we manage, structure, and build our source code must support existing code as well as it supports new code. This proposal provides direct support for existing code by allowing header files to interoperate directly with the introduced import mechanism.
- *You don't pay for what you don't use.* This proposal aims to adopt this principle at several levels. When importing an interface (including a legacy header unit), the cost grows with the portion of the interface used, not with the transitive size of the interface. Only names and macros from imported translation units are made visible, mitigating the risk of accidental name collision.

§ 3. Basics

§ 3.1. Modules

This proposal introduces the notion of a **module**, which represents an encapsulated component within a program. A module comprises a collection of translation units of four kinds (collectively known as **module units**):

- The **interface unit** of a module is a translation unit defining the external interface of the module. They are introduced with the syntax

```
export module module-name ;
```
- A **module partition** is a translation unit that supports semantic import within the same module. Module partitions may be re-exported from the primary interface unit, but need not be. They are introduced with the syntax

```
export module module-name : partition-name ;
```


See [§3.3 Module partitions](#).
- An **implementation unit** of a module is a translation unit providing implementation details within the semantic scope of the module. They are introduced with the syntax

```
module module-name ;
```
- A **legacy header unit** of a module is a translation unit synthesized by the implementation to allow import of a legacy header file. See [§6 Legacy headers](#).

A module that contains any translation units that are not legacy header units must contain exactly one interface unit. There are no other bounds on how many or few translation units it can contain.

Each module depends on a set of other modules; the mechanism for specifying this set is implementation-defined. If a module imports a translation unit owned by a module on which it does not have a dependency, the program is ill-formed.

Rationale:

Requiring module dependencies to be explicitly specified has many advantages:

- It provides the build system the necessary information to link in all used modules, by producing a compilation error if such a build system dependency is missing.
- It allows external enforcement of layering, by requiring layering in the module dependencies in the build system or another tool.
- It allows external enforcement of visibility restrictions (for instance, if certain modules should only be used by a restricted set of other modules, the compiler can help enforce this constraint).

A build tool can choose to compute the set of module dependencies by extracting them from the source code itself, if dependency constraints are not desired for the project.

§ **3.1.1. Module ownership**

Entities declared within a module are *owned* by that module. Declarations owned by distinct modules are distinct entities. The program is ill-formed (no diagnostic required) if two modules export conflicting declarations of the same name; non-exported declarations may coexist so long as a declaration of one such entity does not occur while another such entity is visible.

As a special exception, some entities are never owned by a module even if declared within a module:

- namespaces (namespace members may be owned, but namespace names never are)
- the global replaceable allocation and deallocation functions
- entities first declared within language linkage specifications (`extern "C"` and `extern "C++"`)

Rationale:

Module ownership allows encapsulation of implementation details of modules, so that two translation units in a module can share names internally without risking collisions with other modules.

However, namespaces are still the mechanism by which global uniqueness of names is ensured: names crossing module boundaries (those names that are exported) are still fundamentally a single global resource shared across the program, as we can expect that if a program has two conflicting definitions of a name, those two different meanings will eventually be needed in the same translation unit. Hence such situations are disallowed, so that poor namespace discipline can be caught and remedied early.

§ **3.2. Translation unit structure**

A module unit has the following structure:

module-declaration

import-declaration

import-declaration

...

import-declaration

declaration

declaration

...

declaration

Note that the *module-declaration* appears first, and all *import-declarations* appear before any other *declarations*.

A translation unit that is not a module unit has the same structure, with the leading *module-declaration* omitted.

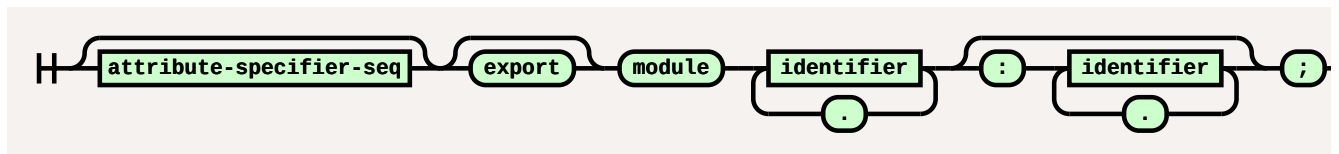
Rationale:

Requiring a specific order provides several advantages:

- It enforces a convention that is near-universally considered to be good style.
- It permits the dependency graph of a module to be quickly determined, without analysing the entire source file, and for the correspondence between source files and modules to likewise be quickly determined.
- Requiring the *module-declaration* to be first ensures the owning module is known before the compiler performs steps that might depend upon it, such as opening output files and generating mangled names.

§ 3.2.1. The *module-declaration*

The general form of a *module-declaration* is:



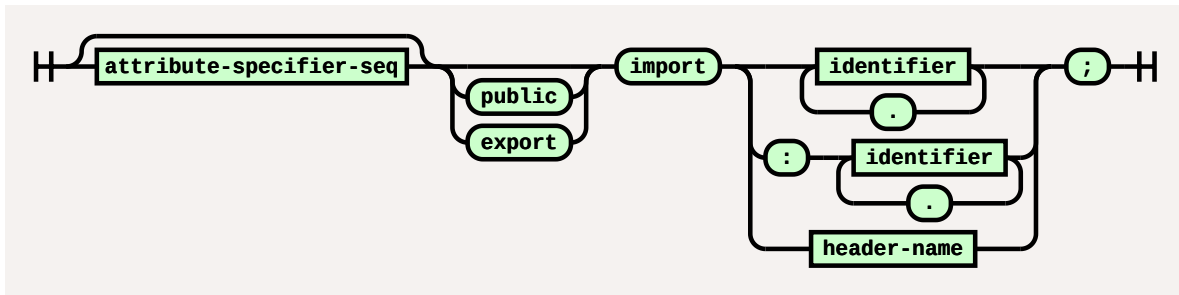
The first dotted sequence of *identifiers* is the *module-name*, which identifies the module owning the translation unit. If present, the second dotted sequence of *identifiers* is the *partition-name*, which identifies the translation unit as being importable within the same module.

If the optional **export** keyword is absent, there shall be no *partition-name*, and *module-declaration* implicitly imports the interface unit of the module. See [§4.2 Transitive visibility of imports](#) for the semantics of imports.

module is a context-sensitive keyword that is recognized only in the formation of a *module-declaration* at the start of a translation unit.

§ 3.2.2. *import-declarations*

The general form of an *import-declaration* is:



An *import-declaration* specifies that the named translation unit is to be imported. Following the `import` is either a *module-name*, identifying the module to be imported, or a `:` and a *partition-name*, identifying a partition of the current module to be imported ([§3.3 Module partitions](#)), or a *header-name*, identifying a legacy header unit to be imported ([§6 Legacy headers](#)).

`import` is a context-sensitive keyword that is recognized only in the formation of an *import-declaration* in the block of *import-declarations* at the start of a translation unit.

The semantics of imports are described in [§4 Imports](#).

§ 3.3. Module partitions

A module partition is a module unit that can be imported into other module units of the same module. Each module partition has a unique name suffix, which is separated from the *module-name* by a colon. Module partitions are an implementation detail of their containing module that is not observable to code outside the module. In particular, a module partition cannot be imported from outside its module.

Module partitions permit the interface of a module to be factored across several files. They also permit module-internal declarations to be moved to a file that is imported but not re-exported by the module's interface unit, while leaving implementation details visible to the interface unit itself. Finally, they permit implementation details to be shared between multiple translation units of a module without putting those implementation details into the module interface unit.

Unlike for a module implementation unit, the *module-declaration* of a module partition does not implicitly import the interface unit of the module. As a consequence, the interface unit of the module may choose to import (and re-export, if it desires) a module partition without creating an import cycle. Alternatively, a module partition may import the interface unit of the module if desired.

EXAMPLE 1

```
export module widget:base;
export class Widget {};
```

```
export module widget:bolt;
import :base;
export class Bolt : Widget {};
```

```
export module widget;
export import :base;
export import :bolt;
void frob(Widget*);
```

```
export module widget:utils;
import widget; // make Widget, Bolt, frob visible
import std.vector;
inline void frob_helper(const std::vector<Widget*> &widgets) {
    for (Widget *w : widgets) frob(w);
}
```

```
module widget;
import :utils;
void frob(Widget *w) {
    frob_helper(children(w));
}
```

Here, the module partitions `:base` and `:bolt` are re-exported by the interface unit, and so their interfaces contribute to the interface of the module. The module partition `:utils` is not exported by the interface unit, and is not part of the module's interface. However, it wishes to refer to symbols declared in the interface unit, and therefore must import the interface unit. The final translation unit in this example is an implementation unit, whose *module-declaration* implicitly imports the interface unit. However, the partition `:utils` must be explicitly imported here to make the name `frob_helper` visible.

`export` declarations in a module partition only have an effect if the module partition is exported by the interface unit. Implementations are encouraged to issue a warning if a module partition that contains an `export` declaration is not re-exported by the module's interface unit.

§ 4. Imports

Importing a translation unit makes its interface available to the importing code. This includes declaration names, semantic effects of declarations, and macros.

The interface of a module can be imported by importing its interface unit, partitions of the current module can be imported by name, and legacy header modules can be imported to import the interface of a legacy header file. All forms of import follow the same rules.

§ 4.1. Import semantics

Import declarations have the following effect:

- The imported translation unit is identified.
- All namespace-scope names and macros exported by that translation unit are made visible in the current translation unit.
- The semantic effects of all declarations in the imported translation unit take effect in the current translation unit.

If the imported translation unit and the current translation unit are owned by the same module, all namespace-scope names and macros from the imported translation unit are made visible in the current translation unit, regardless of whether they are exported.

The semantic effects that are imported from a translation unit include

- whether a class is defined (and if so, its member names),
- whether a default argument is available,
- the existence and definitions of template specializations,
- definitions of constexpr functions,

and so on. Imported semantic effects are said to be **available** in the importing translation unit.

EXAMPLE 2

```
export module widget;  
struct Widget { int get(); };  
export Widget make();
```

```
import widget;  
int f() {  
    auto w = make(); // ok, name 'make' and definition of class Widget are imported  
    return w.get(); // ok, type definition is imported  
}  
Widget w; // error, name 'Widget' is not visible here
```

The behavior is the same no matter whether `struct Widget`'s definition appears before or after the `make` function. For example, this definition of the interface unit of `widget` is exactly equivalent to the one above:

```
export module widget;  
struct Widget;  
export Widget make();  
struct Widget { int get(); };
```

Names and semantic properties are not exported transitively by default.

EXAMPLE 3

```
export module handle:impl;  
struct Impl { int n; };
```

```
export module handle;  
import :impl;  
export using Handle = Impl *;  
export Handle make();
```

```
import handle;  
int f() {  
    Handle h = make();  
    return h->n; // error, definition of Impl is not available here  
}
```

Cyclic imports are disallowed.

Rationale:

We take it as an ideal that the semantics of a translation unit should not depend on the order in which its constituent declarations appear. Therefore, all semantic effects in the translation unit are exported, regardless of whether they occur before or after an `export` declaration. (In the `widget` example, it does not matter whether `struct Widget` is defined before or after `make`.)

Only semantic effects that are in some way reachable from an exported declaration need actually be made available to importers of the translation unit. An implementation can still prune out those effects that are purely internal, such as the definition of a non-exported class that is not made reachable by any exported declaration.

As demonstrated in the `handle` example, it is straightforward to separate out semantic effects that exist only for use by the implementation of the module (including use within the interface unit itself) so they are not visible to consumers of the module.

§ 4.2. Transitive visibility of imports

Three different forms of import are available, providing control over the transitive visibility of names and semantic effects of imported translation units:

```
import foo.bar; // a private import  
public import foo.bar; // a public import  
export import foo.bar; // an exported import
```

§ 4.2.1. Private imports

```
import foo.bar;
```

By default, the imports of a translation unit are not made visible to translation units that import it in any way: names of declarations and macros from the imported translation unit are not transitively made visible, and definitions of entities, default arguments, and so on are not made available transitively. Such an import is known as a ***private import***.

This default is appropriate when an import is intended for the consumption of the translation unit itself and does not form part of its interface: this permits the maintainer of the translation unit to remove imports that they are no longer using, without risk of breaking downstream consumers of the translation unit who are inadvertently (or deliberately) depending on it.

§ 4.2.2. Exported imports

```
export import foo.bar;
```

An import can be preceded by the **export** keyword, forming an ***exported import***. This makes the imported translation unit transitively visible. Importing a translation unit containing an exported import is equivalent to importing that translation unit and also importing the translation unit named by the exported import, except that macros are not implicitly re-exported (see [§5.2 Macro export](#)).

This form of import is appropriate when the interface of one translation unit is intended to form part of the interface of another translation unit by aggregation.

§ 4.2.3. Public imports

```
public import foo.bar;
```

A ***public import*** provides a hybrid between the default importation mode and an exported import. No names or macros from the imported translation unit are made visible in importers of the current translation unit (unless explicitly re-exported), but all the other semantic properties of the imported translation unit do take effect. This permits a translation unit to export an interface that is a modified form of the interface of another translation unit.

EXAMPLE 4

```
module std.cstdlib;
public import <stdlib.h>;
export namespace std {
    using ::atof;
    using ::atol;
    // ...
    using ::div;
    // ...
    using ::div_t; // exported as a complete type because we publicly imported <stdlib.h>
    using ::size_t;
}
#export EXIT_FAILURE
#export EXIT_SUCCESS
#export MB_CUR_MAX
#export NULL
#export RAND_MAX

import std.cstdlib;
void f() {
    std::div_t a = std::div(4, 3); // ok, definition of div_t is available
    div_t a = div(4, 3); // error, names 'div_t' and 'div' are not visible here
}
```

§ 4.3. Preprocessor impact

`module` and `import` declarations affect both the behavior of the C++ language in phase 7 of translation onwards, and the behavior of the C++ preprocessor, as imports may introduce macro names. To support this, a restriction is applied to the initial sequence of *preprocessing-tokens* from which these declarations are derived:

- The terminating semicolon of a *module-declaration* or *import-declaration* shall not be produced by a macro expansion, and shall not be preceded by the expansion of an imported macro.

The preprocessor is expected to identify the initial *module-declaration* and sequence of *import-declarations* as it produces them, and to apply the semantic effects of those declarations at the point of the corresponding semicolon. The preprocessor and compiler proper may share a representation for a precompiled translation unit, or may use distinct representations or some other implementation technique, but must interpret the imported translation unit name as naming the same notional translation unit.

The token following an `import` token can be a *header-name* token, such as an angled string literal token (for example `<foo>`). Such tokens are only formed in special situations (currently, after a `#include` or a `__has_include()`), and the use of a *header-name* token in an *import-declaration* adds one more such situation. As such, after lexing an `import` token that might form part of an *import-declaration*, the following token is lexed as a *header-name* token if possible. If the token after the *header-name* token is not a `;` token, the *header-name* token must be reverted and re-processed as regular non-*header-name* tokens.

Rationale:

The terminating semicolon is required to literally appear within the translation unit source code in order to avoid any ambiguity as to where imported macros become available for use. As an example of the problems that could otherwise arise, consider:

```
// foo exports a macro "#define BAR baz"
#define IMPORT import foo; BAR
IMPORT
```

Depending on whether the effect of the import occurs during or after rescan of the expansion of the `IMPORT` macro, the `BAR` macro from `foo` may or may not be expanded. This is avoided by requiring the `;` (the point at which macros become visible) to not be produced by macro expansion.

We also wish to permit the set of imports of a translation unit to be determined without knowledge of the contents of the imported translation units. In particular, the full set of dependencies should be discoverable (for instance, by a build tool or a non-compiler parser of source code) without the need to consult external files, safe in the knowledge that no macro will (for instance) `#define import`. However, preprocessor action should still be permitted in the import declaration region, to allow constructs such as:

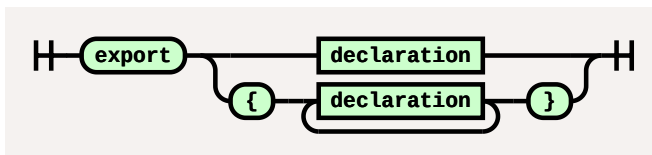
```
#ifdef BUILDING_ON_UNIX
import support.unix;
#else
import support.windows;
#endif
```

To this end, macro expansion before the end of the initial sequence of *import-declarations* is disallowed from expanding an imported macro. (However, imported macros are still visible from the terminating semicolon of the relevant import declaration, as we need to update the preprocessor state immediately in case the first non-import declaration begins with a use of an imported macro.)

§ 5. Exports

§ 5.1. Name export

A namespace-scope declaration can be exported by prefixing it with the `export` keyword:



EXAMPLE 5

```
export struct X { ... };
```

Such a declaration shall declare at least one namespace-scope name, and all namespace-scope names declared within an `export` declaration (including names transitively declared within a namespace inside the declaration) are exported. The

export rules apply only to names, but apply uniformly to all kinds of names (including names of *using-declarations* and other kinds of declarations that do not introduce new entities).

The name of a namespace is exported if it is ever declared within an *export-declaration*. A namespace name is also implicitly exported if any name within it is exported (recursively).

EXAMPLE 6

```
export module namespaces;
export namespace A { // A is exported
    int n; // A::n is exported
}
namespace B {
    export int n; // B::n is exported and B is implicitly exported
}
namespace C {
    int n;
}
export namespace C {} // C is exported, C::n is not
```

Names not declared at namespace scope (for example, names of class members, enumerators of local or scoped enumerations, and class-scope friend declarations) are visible if the enclosing definition is available (that is, if the semantic effect of defining the class has either occurred in the current translation unit or has been imported from another translation unit).

EXAMPLE 7

```
export module A;
class NotExported {
    friend void munge(NotExported);
    friend void frob(NotExported);
    void badger();
};
void frob(NotExported);
export NotExported make();

import A;
int main() {
    auto x = make();
    x.badger(); // OK, definition of 'NotExported' is visible
    munge(x);   // OK, found by ADL inside definition of 'NotExported'

    NotExported ne; // ill-formed: NotExported not visible
    auto *fp = &frob; // ill-formed: frob is not visible, class-scope
                      // declaration only visible to ADL
}
```

An exported name shall not have internal linkage. Exported namespace-scope variables with `const`-qualified types do not implicitly have internal linkage.

§ 5.2. Macro export

Note: This section is a separable, pure extension to the rest of this proposal, and we would consider it feasible to relegate this feature to a compatibility Annex or to a separate TS that is not intended to ever be merged into the IS. In the absence of a macro export syntax, only legacy header imports (`import <header>;` and `import "header.h";`) can result in macro import; regular *import-declarations* cannot.

Macros can be exported using the `#export` directive.

EXAMPLE 8

```
#define F00(x) ((x) + 1)
#export F00
```

The traditional C preprocessor assumes that `#defines` and `#undefs` occur in a single linear order, but that is no longer the case once macros can be exported and imported. To resolve conflicts between macro definitions across translation units, the following rules are used:

- Each definition and undefinition of a macro is considered to be a distinct entity.
- Such entities are visible if they are from the current translation unit, or if they were exported from a translation unit that has been imported.
- A `#define X` or `#undef X` directive *overrides* all definitions of `X` that are visible at the point of the directive.
- A `#define` or `#undef` directive is *active* if it is visible and no visible directive overrides it.
- A set of macro directives is *consistent* if it consists of only `#undef` directives, or if all `#define` directives in the set define the macro name to the same sequence of tokens (following the usual rules for macro redefinitions).
- If a macro name is used and the set of active directives is not consistent, the program is ill-formed. Otherwise, the (unique) meaning of the macro name is used.

EXAMPLE 9

Suppose:

- `<stdio.h>` defines a macro `getc` (and exports its `#define`)
- `<cstdio>` imports the `<stdio.h>` module and undefines the macro (and exports its `#undef`)

The `#undef` overrides the `#define`, and a source file that imports both modules (in any order) will not see `getc` defined as a macro.

Note that the effect of a sequence of imports does not depend on the relative ordering of those imports, but nonetheless permits macros to be overridden as in traditional use of the preprocessor.

`#export` also supports macro name globbing; the token sequence after `#export` is required to be an alternating sequence of *identifier* or *pp-number* tokens and `*` tokens, where a `*` matches any sequence of characters in a macro name.

EXAMPLE 10

```
import <inttypes.h>;
#export SCN*
#export PRI*
```

```
import <limits.h>;
#export *_MIN
#export *_MAX
```

During rescan of an expansion of a macro that was exported from another translation unit, all macros exported from that translation unit are visible.

EXAMPLE 11

```
export module a;
#define F00 BAR
#define BAR BAZ
#export *
```

```
export module b;
import a;
#export F00
```

```
import b;
int BAR; // unchanged
int F00; // expands to "int BAZ;" even though macro BAR is not visible here
```

Rationale:

Macros form part of the interface of many modern C++ libraries. Any system that seeks to support exporting C++ interfaces must therefore provide a mechanism to allow macros to be exported. This could instead be accomplished by forcing the macros into a separate, textually-included file, but doing so forces an awkward artificial separation between portions of the same interface, prevents existing header files from being transparently converted into equivalent importable translation units, and makes it error-prone to maintain a library that provides both a header file interface and an importable interface.

§ 6. Legacy headers

Legacy header support permits existing header files to be used from modular code without sacrificing modularity: names do not leak into the imported header file, compilation performance is not sacrificed by recompiling the same header files on every inclusion, and users do not need to resort to the preprocessor to access the interfaces of non-modular libraries.

§ 6.1. Legacy header units

Legacy header units are translation units synthesized by the implementation to wrap an existing header file as part of a module. Each legacy header unit has as its name a *header-name* that identifies the wrapped file. The synthesized legacy

header unit comprises:

```
export module header-name ;
export extern "C++" {
#include header-name
#export *
}
```

(where the *header-name* here is for exposition only; a *header-name* is not a valid module name).

The entire contents of the header file are exported, including all names, macros, and semantic effects. The entities within the header are treated as not being owned by any module.

As a special exception, if the module declares an entity with internal linkage (or an entity whose name involves such an entity), the export is not ill-formed, but the program is ill-formed if the entity is odr-used by the importer (including within a template instantiation whose point of instantiation is outside the legacy header unit).

The *header-name* in a header import declaration is first looked up as the name of a legacy header unit of the current module (if any). If that lookup fails, it is looked up in direct dependency modules; if it is found, it shall only be found in one such module. Finally, if that lookup also fails, the header import declaration is conditionally-supported. (An implementation is permitted, but not required, to translate the named header into a legacy header unit, or to perform lookup in additional modules, to satisfy the import.)

§ 6.2. #include translation

Within a legacy header unit, a preprocessor directive

```
#include FILE
```

is interpreted as including a file comprising `import FILE`; if `FILE` is a *header-name* naming another legacy header unit owned by the current module or one of its direct dependencies. Unlike a regular `import`, such a `#include` may appear anywhere in the translation unit.

If the *header-name* does not correspond to a legacy header unit of the current module or one of its direct dependencies, it is textually included. This is appropriate for headers that are intended to interact with the preprocessor in ways more complex than providing macros as output (for instance, files that depend on the macro state at the point when they are imported), or for files that are intended to generate raw tokens rather than an encapsulated set of declarations.

Rationale:

It would be undesirable and inefficient for the translation unit generated for a legacy header unit to contain the contents of the transitive closure of `#includes` in its header; a translation unit importing many such headers would transitively import a great many copies of the same declarations and definitions. In order to approximate the ideal that each entity has only one definition in the entire program, `#included` headers encountered while building legacy module units should be treated as imports where possible.

As `#include` may also be used to include files that fundamentally intend to have a textual effect on the compilation, translation from header file `#includes` to `import` is not fully automated, and some module must nominate the header file as a legacy header unit for the translation to occur.

§ 6.3. Compilation model

Note: This section is purely informative and is not part of the proposal.

Multiple implementation strategies are possible for supporting legacy headers. The Clang implementation of header module support (which has been in use in production by multiple parties for several years) has validated the feasibility of both a cached in-process compilation-on-demand model and an ahead-of-time separate compilation model. The GCC implementation of the Modules TS has gained experience with allowing the compiler to "call back" into the build system to request module dependencies, and this system also seems feasible for implementing legacy header unit support.

Possibly the most straightforward approach would be to compile the module interface unit and the legacy header units as part of a single compilation action, passing the names of all relevant files to the compiler together; this compilation action would produce a binary representation of the module interface (the emerging convention is to call this a "BMI" per [\[GCCModules\]](#)).

EXAMPLE 12

Source files:

```
// widget.h: some pre-existing widget library
// ...
```

```
// gromit.h: some specific widget
#include "widget.h" // transparently imports legacy header unit for "widget.h"
// ...
```

```
// widget.cppm
// provide a module interface that exposes only widget.h and not gromit.h
export module widget;
export import "widget.h";
```

```
// legacy_user.cpp
#include "gromit.h" // transparently imports legacy header unit for "gromit.h"
// ...
```

```
// modern_user.cpp
import widget;
// ...
```

Compilation commands:

```
$ cc -fmodules widget.cppm widget.h gromit.h --precompile -o widget.pcm
$ cc -fmodules -fmodule-file=widget.pcm legacy_user.cpp -c -o legacy_user.o
$ cc -fmodules -fmodule-file=widget.pcm modern_user.cpp -c -o modern_user.o
```

Compiling all the legacy header units of a module as part of the same build action allows the compiler to resolve the dependencies between them (compiling them in topological order) without any need for explicit dependencies to be inferred between the header files.

§ 7. Low-level design details

§ 7.1. Templates and two-phase name lookup

When a template is imported from another translation unit and instantiated, it must have an appropriate set of names and semantic properties available for the instantiation to use. Templates generally rely on names and semantic properties from two sources:

- those provided alongside the template, and
- those provided by the code directly or indirectly instantiating the template.

In general, the semantic properties (and, for argument-dependent lookup, names) provided alongside the template may include properties introduced either before or after the template is defined.

Note that if one template instantiation triggers another, the inner template instantiation may rely on names and properties provided by the outer template instantiation and on those provided at its point of instantiation. Therefore, we define the visibility rules for template instantiation as follows:

Within a template instantiation, the *path of instantiation* is a sequence of locations within the program, starting from the ultimate point of instantiation, via each intervening template instantiation, terminating at the instantiation in question. Names are visible and semantic properties are available within template instantiations if they would be visible or available at any point along the path of instantiation, or (for points outside the current translation unit) would be visible or available at the end of the translation unit containing the relevant point of instantiation.

EXAMPLE 13

This example is borrowed from **[temp.dep.res]** in [\[N4720\]](#), adjusted suitably for this proposal.

```
export module A;
export template<typename T>
void f(T t) {
    t + t; // #1
}
```

```
export module B;
import A;
export template<typename T, typename U>
void g(T t, U u) {
    f(t); // #2
}
```

```
export module C;
import <string>;
import B;
export template<typename T>
void h(T t) {
    g(std::string{ }, t); // #3
}
```

```
import C;
void i() {
    h(0); // #4
}
```

The instantiation of `f<std::string>` has a path of instantiation comprising:

- Point #4 within `i()`
- Point #3 within `h<int>(int)`
- Point #2 within `g<std::string, int>(std::string, int)`
- Point #1 within `f<std::string>(std::string)`

Therefore, semantic properties available at point #4, as well as those available in the module interface units of modules `A`, `B`, and `C`, are available within the instantiation of `f<std::string>`. Because the `<string>` legacy header unit is imported into the interface unit of module `C`, declarations from that header are visible in the instantiation, so the expression `t + t` at line #1 is valid.

Rationale:

This rule makes a necessary and sufficient set of names and properties visible. Each translation unit contributing a template involved in the instantiation could provide one of the types or functions that is intended to be used by the instantiation, so must be included. And the instantiation could be performed with only this set of translation units involved, so any other types and functions are not guaranteed to be available to an instantiation.

§ 8. Acknowledgements

Thanks to David Blaikie, Chandler Carruth, Daniel Dunbar, Duncan Exon Smith, David Jones, Thomas Köppe, Bruno Cardoso Lopes, and Vassil Vassilev for comments on early drafts of this proposal.

Thanks again to all those involved in the Modules TS, and particularly Gabriel Dos Reis, for exploring the modules design space and providing a basis for this paper.

Thanks to Doug Gregor et al for the initial design of Clang’s modules implementation, on which the legacy header support in this paper is based.

§ Index

§ Terms defined by this specification

[available](#), in §4.1

[interface unit](#), in §3.1

[module unit](#), in §3.1

[exported import](#), in §4.2.2

[legacy header unit](#), in §3.1

[private import](#), in §4.2.1

[implementation unit](#), in §3.1

[module](#), in §3.1

[public import](#), in §4.2.3

[module partition](#), in §3.1

§ References

§ Informative References

[GCCModules]

Nathan Sidwell. [GCC C++ Modules wiki page](#). URL: <https://gcc.gnu.org/wiki/cxx-modules>

[N4697]

Barry Hedquist. [NB Comments, ISO/IEC PDIS 21544, C++ Extensions for Modules](#). URL: <https://wg21.link/n4697>

[N4720]

Gabriel Dos Reis. [Working Draft, Extensions to C++ for Modules](#). URL: <http://wg21.link/n4720>

[P0273R0]

Richard Smith, Chandler Carruth, David Jones. [Proposed modules changes from implementation and deployment experience](#). 12 February 2016. URL: <https://wg21.link/p0273r0>

[P0678R0]

John Lakos. [Business Requirements for Modules](#). URL: <https://wg21.link/p0678r0>

[P0713R0]

Daveed Vandevor. [Identifying Module Source Code](#). URL: <https://wg21.link/p0713r0>

[P0774R0]

Nathan Sidwell. [Module-decl location](#). URL: <https://wg21.link/p0774r0>

[P0775R0]

Nathan Sidwell. [module partitions](#). URL: <https://wg21.link/p0775r0>

[P0795R0]

Simon Brand, Neil Henning, Michael Wong, Christopher Di Bella, Kenneth Benzie. [From Vulkan with love: a plea to reconsider the Module Keyword to be contextual](#). URL: <https://wg21.link/p0795r0>

[P0841R0]

Bruno Cardoso Lopes, Adrian Prantl, Duncan P. N. Exon Smith. [Modules at scale](#). URL: <https://wg21.link/p0841r0>